

Operation TrackPoint

A full-lifecycle DevOps hackathon challenge

Take a "works on my laptop" parcel-tracking application and turn it into a secure, automated, observable, recoverable and scalable production system, from an empty server all the way to Kubernetes and a live incident drill.

Item	Detail
Role you are playing	Junior-to-mid DevOps / Platform Engineer (first and only infra hire)
Format	11 progressive stages + final incident challenge + evaluation
Estimated effort	120–180 hours (8–12 weeks at 12–15 h/week)
Budget	USD 30 / month for all infrastructure (domain excluded)
Outcome	A portfolio-grade public repository, documentation set, and demo
Issued	September 2026

How to Use This Document

This is not a tutorial. It is an assignment written the way a real company would write one: goals, constraints, acceptance criteria and a deadline. **You** decide how to meet them. There is no single correct solution; there are only solutions you can justify and defend.

Rules of engagement

1. **No copy-paste solutions.** You may read documentation, blog posts and other people's code, but every file you commit must be something you can explain line by line.
2. **Everything is code.** If you did something by hand on a server, it does not count until it is reproducible from the repository.
3. **Break things deliberately.** Each stage has a "Break it on purpose" section. Skipping it defeats the purpose of the challenge.
4. **Write as you go.** Keep a `docs/learning-log.md`: what you tried, what failed, why, and what fixed it. This log is part of your evaluation.
5. **Timebox before asking for help.** Spend at least 45 minutes on a problem (reading logs, docs, experimenting) before opening a hint. Spend at least 90 minutes before asking your mentor.
6. **Never commit secrets.** Not even "just for a minute". A single leaked credential is an automatic incident you must handle (see Stage 7).

How to ask your mentor for help

When you are stuck, ask for help at the lowest level that gets you moving. Tell your mentor which level you want:

Level	What you get	When to use it
L1 — Nudge	A question that points you in the right direction	You know roughly where the problem is but not what to look at
L2 — Concept	An explanation of the concept or mechanism involved, no code	You don't understand why something behaves the way it does
L3 — Pointer	The specific tool, flag, config key or doc page to look at	You understand the concept but can't find the lever
L4 — Walkthrough	Step-by-step commands with explanation	Only after L1–L3, or when blocked for more than a day

MENTOR — Starter app & chaos scripts on request

This challenge specifies the application as a contract (Section 3), not as finished code. You may write it yourself in any language (recommended: ~300–500 lines in Node.js, Python or Go), or ask your mentor to generate a **starter repository with deliberately planted flaws** that mimic what a real dev team would hand you.

For Stage 11 you can also ask your mentor for a **sealed chaos script** that injects a random failure without telling you which one. That is the most realistic way to practise incident response.

Document map

Part	Contents	Use it for
Part I — The Brief	Sections 1–17: scenario, requirements, deliverables, scoring	Understanding what “done” means. Re-read before every stage.
Part II — The Stages	Stages 1–10 with tickets, requirements, checkpoints, hints	Your week-by-week work plan
Part III — Incidents	Stage 11: the final incident challenge	Proving you can operate what you built
Part IV — Assessment	DevOps Engineer Evaluation Checklist + oral defence	Measuring yourself honestly
Appendices	Repo layout, templates, resources, progress tracker	Reference while you work

Contents

How to Use This Document	2
Rules of engagement	2
How to ask your mentor for help	2
Document map	3
PART I — THE BRIEF	6
1. Problem Statement	6
2. Company & Project Scenario	6
3. Application Requirements	7
4. Initial Infrastructure Situation	8
5. Business Requirements	9
6. Technical Requirements	9
7. Security Requirements	10
8. Performance Requirements	11
9. Deployment Requirements	12
10. Monitoring Requirements	12
11. Backup and Disaster-Recovery Requirements	12
12. CI/CD Requirements	13
13. Scaling Requirements	13
14. Failure Scenarios You Must Handle	14
15. Final Deliverables	15
16. Evaluation Criteria	15
17. Bonus Challenges	16
PART II — THE STAGES	18
Stage 1 — Foundation	19
Stage 2 — Containerization	22
Stage 3 — Production Deployment	25
Stage 4 — CI/CD	28
Stage 5 — Infrastructure as Code	31
Stage 6 — Observability	34
Stage 7 — Security	37
Stage 8 — Reliability	40
Stage 9 — Scaling	43
Stage 10 — Kubernetes	46
PART III — STAGE 11: FINAL INCIDENT CHALLENGE	49
How to run the game day	49
Incident cards	49
The final boss: compound incident	54
PART IV — DEVOPS ENGINEER EVALUATION CHECKLIST	55
Beginner	55
Intermediate	55
Advanced	56
Production-ready	56
How the components work together (oral defence)	56
Appendix A — Suggested Repository Layout	58
Appendix B — Templates	59
Runbook template	59
Post-mortem template (blameless)	59
Architecture Decision Record (ADR) template	59
Suggested ADR topics	60
Appendix C — Study Resource Library	61
Appendix D — Progress Tracker	62

(If the table of contents appears empty, right-click it in Word and choose "Update Field".)

PART I — THE BRIEF

Part I is the full assignment as the company would give it to you. Every requirement here is tested somewhere in the stages or in the final evaluation. Treat numbered IDs (for example **SEC-04**) as acceptance criteria: your final README should map each ID to the file, dashboard or document that proves it is met.

1. Problem Statement

Harborline Logistics has built **TrackPoint**, a parcel-tracking platform used by retail clients to create shipments and by end customers to track deliveries. It currently runs on a single, hand-configured server that nobody fully understands. The contractor who set it up has left. Last month the platform was down for six hours because the disk filled up, and nobody noticed until customers complained on social media.

The company has signed three pilot retail clients who go live in **eight weeks**. The contracts include an availability commitment, data-protection obligations for customer personal data, and a requirement that tracking pages stay fast during the holiday peak.

CONSTRAINT — Your mission

Design, build, secure, automate, monitor and document a production platform for TrackPoint, starting from nothing, that the company can trust with paying customers, and that a new engineer could take over from your documentation alone.

2. Company & Project Scenario

The company

- **Harborline Logistics:** a 40-person last-mile delivery company operating in three cities.
- Engineering team: two backend developers, one frontend developer, and now you, the first infrastructure hire.
- The developers are good at features but have never run production systems. They push directly to **main** and deploy by copying files over SSH.
- Management cares about three things: **don't go down during business hours**, **don't leak customer data**, and **don't blow the budget**.

The people you work with

Person	Role	What they want from you
Amara Osei	CTO (your manager)	Reliability, a clear plan, weekly status updates, and no surprises
Dev team	Backend + frontend developers	Fast feedback on pull requests; the ability to deploy without asking you
Jonas Keller	Operations Manager	Tracking pages that always work; notice before customers do
Legal / compliance	External advisor	Evidence that personal data is protected, backed up and access-controlled
Finance	Part-time accountant	A predictable monthly bill under USD 30

Timeline

Week	Milestone	Stages
1	Foundation: server, repo, working app locally	1
2	Everything containerised and running with Compose	2
3	Public HTTPS endpoint on the company domain	3
4	Automated pipeline: no more manual deploys	4
5	Rebuildable from code; monitored and alerting	5, 6
6	Security hardening and audit	7
7	Backups proven; zero-downtime deploys; scaling for peak	8, 9
8	Kubernetes migration (pilot) + incident game day + handover	10, 11

3. Application Requirements

TrackPoint consists of five components. You may implement the application code yourself, keep it deliberately small, or request a starter repository from your mentor. **The application is not what is being graded; how you run it is.** It must, however, honour the contract below, because your infrastructure depends on it.

Component	Purpose	Operational notes
web	Static single-page frontend (tracking page + client dashboard)	Built at CI time into static files; served by a web server, never by a dev server
api	REST API: parcels, tracking events, client accounts, reports	Stateless; horizontally scalable; configured only via environment variables
worker	Background jobs: label PDF generation, notification e-mails, nightly report aggregation	Consumes jobs from a Redis queue; must survive restarts without losing jobs
postgres	Primary relational database (PostgreSQL 16+)	Holds personal data (names, addresses, phone numbers) — must be protected and backed up
redis	Job queue, cache and session store	Cache data is disposable; queue data is not — think about the difference

API contract (the parts your infrastructure depends on)

Endpoint	Behaviour	Why it matters to you
GET /healthz	Returns 200 if the process is alive. Does not check dependencies.	Liveness probe / restart decisions
GET /readyz	Returns 200 only if DB and Redis are reachable; 503 otherwise.	Readiness / load-balancer routing decisions

Endpoint	Behaviour	Why it matters to you
GET /metrics	Prometheus-format metrics: request count, latency histogram, errors, DB pool usage.	Monitoring; must not be public
GET /api/version	Returns git commit SHA, build time and environment name.	Proving which version is deployed; rollback verification
POST /api/parcels	Creates a parcel (authenticated client API key).	Write path; tests DB and queue
GET /api/parcels/{id}	Public tracking lookup.	Hot read path; latency SLO applies
POST /api/parcels/{id}/events	Adds a tracking event (scanner devices).	High-volume write path during peak
GET /api/reports/daily	Aggregated statistics. Deliberately expensive (heavy SQL).	Caching and resource-limit experiments
POST /api/admin/*	Admin operations. Requires admin token.	Must never be reachable from the public internet without controls

Application behaviour requirements

- **APP-01** All configuration comes from environment variables (12-factor). No config files baked into images with real values.
- **APP-02** Logs are written to stdout/stderr as single-line JSON including timestamp, level, request ID, route, status and duration.
- **APP-03** The API handles **SIGTERM** gracefully: stops accepting new connections, finishes in-flight requests (up to 20 s), then exits.
- **APP-04** Database schema changes are applied by a separate migration command, never automatically on API start-up.
- **APP-05** The API propagates or generates an **X-Request-ID** header so a request can be traced through proxy, API and worker logs.
- **APP-06** The app has at least: unit tests, one integration test that uses a real PostgreSQL, and a smoke-test script that can run against any environment URL.

4. Initial Infrastructure Situation

Here is what the departing contractor left behind. You will **replace** all of it. In Stage 1 you will write a risk register for this setup; the audit is part of your grade.

Area	Current state ("legacy box")
Server	One VPS, Ubuntu 20.04 (end of standard support), 2 vCPU / 2 GB RAM, never updated. Root login with a password over SSH on port 22.
Running the app	API started by hand inside a tmux session with nohup . After a reboot someone has to log in and start it again.
Database	PostgreSQL installed via apt, listening on 0.0.0.0:5432 , superuser password is postgres . The app connects as the superuser.

Area	Current state ("legacy box")
Redis	Installed via apt, bound to all interfaces, no password.
Secrets	A <code>.env</code> file was committed to the Git repository eight months ago and later "deleted" in a new commit.
TLS	A certificate obtained manually 11 months ago. Nobody knows whether renewal works.
Deploys	<code>scp</code> of a zip file, then restart the tmux session. No record of what version is running.
Logs	Written to a file in the app directory; never rotated. Disk is at 87%.
Backups	A cron job that runs <code>pg_dump</code> to the same disk. Nobody has ever tried to restore.
Monitoring	None. Outages are reported by customers.
Firewall	Disabled.
Documentation	A sticky note with the root password, reportedly.

NOTE — You don't need the real legacy box

You are building from scratch. Optionally, recreate a small version of the legacy box in a local VM during Stage 1 so you can practise auditing and migrating a badly configured server. This is excellent interview material.

5. Business Requirements

- **BUS-01** Pilot clients go live in 8 weeks. Stages 1–9 must be complete by then; Stage 10 is a follow-up pilot.
- **BUS-02** Monthly availability of the public tracking endpoint $\geq 99.5\%$ (≈ 3 h 39 min of downtime per month).
- **BUS-03** Developers must be able to deploy several times a week during business hours **without customer-visible impact**.
- **BUS-04** Total infrastructure cost $\leq$ **USD 30/month**. You must produce a cost breakdown.
- **BUS-05** Customer personal data must be protected in transit and at rest in backups, and access to it must be limited and auditable.
- **BUS-06** Losing more than **1 hour** of shipment data is unacceptable (RPO). The platform must be restorable within **2 hours** of total server loss (RTO).
- **BUS-07** A new engineer must be able to operate the platform using only your documentation (the "bus factor" requirement).
- **BUS-08** The holiday peak is expected to bring **3× normal traffic**. The platform must handle it without emergency manual work.

6. Technical Requirements

ID	Requirement
TEC-01	Linux servers running a currently supported LTS distribution (recommended: Ubuntu 24.04 LTS or Debian 12).

ID	Requirement
TEC-02	Source code in Git with a protected <code>main</code> branch; all changes via pull request with passing checks.
TEC-03	Every component runs in a container. Images are built by CI, never on the production server.
TEC-04	Images are tagged with an immutable identifier (commit SHA). The tag <code>latest</code> is never used in production.
TEC-05	Local development and production use the same images, configured differently via environment.
TEC-06	All traffic enters through a reverse proxy that terminates TLS. No application container publishes a port to the host's public interface.
TEC-07	Two environments: <code>staging</code> and <code>production</code> , isolated from each other (separate databases, separate secrets).
TEC-08	Servers are provisioned with Infrastructure as Code (Terraform or OpenTofu) and configured with a configuration-management tool (Ansible recommended).
TEC-09	Configuration drift is detectable: re-running your IaC/config-management against a healthy server changes nothing.
TEC-10	Persistent data lives in named volumes or dedicated storage with a documented lifecycle; containers are disposable.
TEC-11	DNS for the company domain is managed as code where your provider allows it.
TEC-12	A Kubernetes deployment (k3s, kind or a managed service) of the same application, using the same images, for Stage 10.

CONSTRAINT — Resource and budget constraints

Recommended footprint: **two small VPS instances** (e.g. 2 vCPU / 2–4 GB RAM each) from a budget provider, or one slightly larger instance plus a free-tier object storage bucket for backups.

The observability stack must not starve the application. You must measure and document its memory footprint.

No budget at all? Use local VMs (Multipass, Vagrant/VirtualBox) and a free dynamic-DNS domain. Document every place where your local setup differs from real production and what you would do differently.

7. Security Requirements

ID	Requirement
SEC-01	SSH: key-based authentication only; root login disabled; named personal user with sudo; brute-force protection enabled.
SEC-02	Host firewall: default deny inbound. Only 80/443 open to the world; SSH restricted as tightly as practical.
SEC-03	PostgreSQL and Redis are unreachable from the internet (verify from an external machine and include the evidence).

ID	Requirement
SEC-04	No secrets in Git, in images, in CI logs or in <code>docker inspect</code> output where avoidable. The leaked legacy <code>.env</code> is treated as compromised: every credential in it is rotated.
SEC-05	Secrets are injected at runtime from a secret store (CI secrets, SOPS/age-encrypted files, Vault, or a cloud secret manager). Justify your choice.
SEC-06	Least privilege in the database: the application user cannot create/drop tables; a separate migration role owns the schema; a read-only role exists for reporting.
SEC-07	Containers run as non-root users, with unnecessary Linux capabilities dropped and a read-only root filesystem where the app allows it.
SEC-08	CI scans images and dependencies for vulnerabilities and fails the build on critical findings that have a fix available.
SEC-09	CI scans for committed secrets on every pull request.
SEC-10	TLS 1.2+ only, automatic certificate renewal, HSTS and standard security headers. Target grade A on an external TLS test.
SEC-11	Admin endpoints and internal dashboards (Grafana, Prometheus, etc.) are not publicly accessible without authentication; preferably not publicly routable at all.
SEC-12	Automatic installation of OS security updates, with a documented reboot policy.
SEC-13	The CI/CD deploy credential can deploy and nothing more (it cannot, for example, get an interactive root shell).
SEC-14	Rate limiting on the public API to blunt abuse and credential stuffing.

8. Performance Requirements

Metric	Target	Measured how
p95 latency, <code>GET /api/parcels/{id}</code>	< 300 ms at 50 req/s sustained	Load test + Prometheus histogram
p95 latency, <code>GET /api/reports/daily</code>	< 800 ms (after caching)	Load test
Peak throughput	150 req/s mixed traffic for 10 minutes	Load test (k6, Locust or similar)
Error rate	< 0.5% 5xx at all loads above	Proxy + app metrics
Host resource headroom	CPU < 70% and RAM < 80% at 50 req/s	Node metrics
API memory	Each API instance runs within a 256 MB memory limit	Container limits + metrics
Cold start	A new API instance is ready to serve within 15 s	Readiness timing

HINT — On load testing

Never load-test production from the production server itself; the load generator competes for the same CPU and the numbers become meaningless. Run it from another machine and record where it ran.

9. Deployment Requirements

- **DEP-01** One command or one merge deploys. Nobody SSHes into a server to deploy.
- **DEP-02** The **same image** that passed tests in staging is promoted to production. No rebuilds between environments.
- **DEP-03** Deploys during business hours cause **no more than 0.1% failed requests** while a continuous load test runs (target: zero).
- **DEP-04** Database migrations are backward compatible for one release (expand → migrate → contract).
- **DEP-05** A failed deploy is detected automatically (health/smoke checks) and does not leave production broken.
- **DEP-06** Rollback to the previous version completes within **5 minutes** and is a documented, tested procedure.
- **DEP-07** Every deploy is recorded: who, what version, when, and result (a deploy log, Git tag, or release record).
- **DEP-08** Production deploys require an explicit approval step; staging deploys are automatic.

10. Monitoring Requirements

- **MON-01** Metrics collected for hosts, containers, the API (RED: rate, errors, duration), PostgreSQL, Redis and the reverse proxy.
- **MON-02** Dashboards: one "service overview" for humans at a glance, and one per layer (host, containers, database).
- **MON-03** Centralised, searchable logs from all containers with at least 14 days retention within your disk budget.
- **MON-04** An external uptime check from outside your infrastructure (explain why it must be external).
- **MON-05** Alerts routed to a real channel you will see (e-mail, Telegram, Discord, Slack or ntfy) for at least: service down, 5xx rate, p95 latency, disk > 80%, memory pressure, container restart loops, certificate expiry < 14 days, and **backup not successful in the last 26 hours**.
- **MON-06** Every alert has a runbook link explaining what it means and the first three things to check.
- **MON-07** Alerts are actionable: no alert that fires daily and gets ignored. Document how you tuned thresholds.
- **MON-08** SLO tracking for BUS-02 with an error-budget view.

11. Backup and Disaster-Recovery Requirements

ID	Requirement
BAK-01	RPO ≤ 1 hour for PostgreSQL. (A nightly dump alone does not meet this; think about continuous archiving or frequent incremental backups.)
BAK-02	RTO ≤ 2 hours from "the production server is gone" to "customers can track parcels again".

ID	Requirement
BAK-03	Backups are encrypted before they leave the server and stored off-site (different provider or at least a different failure domain).
BAK-04	Retention: 7 daily, 4 weekly, 3 monthly. Old backups are pruned automatically.
BAK-05	Restores are tested automatically at least weekly (restore to a scratch database and run a sanity query).
BAK-06	Backup success and freshness are monitored and alerted on (see MON-05).
BAK-07	A documented and timed DR drill: destroy production, rebuild from code + backups, record the actual RTO and RPO achieved.
BAK-08	Decide and document what else needs backing up besides the database (and what deliberately does not).

12. CI/CD Requirements

ID	Stage of pipeline	Requirement
CI-01	Pull request	Lint, unit tests, integration tests against a real PostgreSQL service, Dockerfile lint, secret scan.
CI-02	Pull request	Build the image and scan it; post results on the PR; block merge on failure.
CI-03	Merge to main	Build once, tag with commit SHA, push to a container registry.
CI-04	Merge to main	Deploy automatically to staging, run migrations, run smoke tests.
CI-05	Release	Manual approval, then deploy the same image to production, smoke test, auto-rollback on failure.
CI-06	All	Full PR pipeline under 10 minutes using dependency and layer caching.
CI-07	All	Pipeline definitions are versioned, reviewed and reusable (no copy-pasted jobs across workflows).
CI-08	All	Secrets never printed to logs; third-party actions pinned to a specific version or commit.
CI-09	Scheduled	A nightly or weekly job rebuilds images to pick up base-image security patches and re-scans them.

13. Scaling Requirements

- **SCL-01** Run at least **3 API replicas** behind a load balancer with active health checks.
- **SCL-02** The API is fully stateless: killing any replica loses no sessions or data.
- **SCL-03** Workers scale independently of the API; queue depth is a visible metric.
- **SCL-04** Expensive reads (`/api/reports/daily`) are cached in Redis with a deliberate TTL and invalidation strategy.
- **SCL-05** Database connections are pooled so that $N \text{ replicas} \times \text{pool size}$ can never exceed PostgreSQL's connection limit.

- **SCL-06** Every container has CPU and memory limits based on measurement, not guesses.
- **SCL-07** Demonstrate horizontal scale-out across **two hosts** (Compose + external load balancer, or Kubernetes).
- **SCL-08** Produce a capacity plan: at what traffic level does each component become the bottleneck, and what is the next step?

14. Failure Scenarios You Must Handle

Your platform will be judged by how it behaves when things go wrong. For each scenario below you must either **prevent** it, **detect** it (alert), **survive** it (automatic recovery), or have a **runbook** for it. Record which applies and prove it with a test.

#	Scenario	Minimum expectation
F1	An API container crashes repeatedly after a deploy	Detected, deploy rolled back automatically
F2	The production server reboots unexpectedly	Everything returns by itself, in the correct order
F3	The disk fills up	Alert fires well before 100%; logs and images cannot fill it unbounded
F4	A container leaks memory	Contained by limits; restarted; alert on restarts
F5	PostgreSQL is unavailable for 2 minutes	API reports not-ready, recovers without restart; no data corruption
F6	TLS certificate renewal silently fails	Alert $\geq$ 14 days before expiry
F7	The nightly backup produces an empty or corrupt file	Detected by restore test; alert
F8	A bad migration is deployed	Old version still works during rollout; rollback path documented
F9	The CI pipeline breaks (expired token, bad cache, flaky test)	Diagnosed from logs; nothing half-deployed
F10	A secret is committed to Git	Detected by scanning; rotation runbook executed
F11	CPU spike from an expensive endpoint	Rate limits / caching / limits keep the rest of the service healthy
F12	DNS change or IP change after rebuilding a server	TTL strategy documented; cut-over without long outage
F13	Container network misconfiguration (service cannot reach Redis)	Readiness fails, traffic not routed to broken instance
F14	Total loss of the production server	Rebuilt within RTO from code + off-site backups
F15	Suspicious activity (unknown process, unexpected outbound traffic)	Detected; incident response runbook; forensic notes

15. Final Deliverables

#	Deliverable	Acceptance
D1	Public Git repository (or monorepo) with application, Dockerfiles, Compose files, IaC, config management, pipelines, K8s manifests	A stranger can follow the README and bring up staging
D2	Architecture diagram(s): network, request flow, deployment flow, backup flow	Up to date with the final system; includes ports and trust boundaries
D3	Requirements traceability matrix: every ID in Part I → evidence	No ID left unmapped; gaps explicitly acknowledged
D4	Runbooks: deploy, rollback, restore, DR rebuild, certificate issues, disk full, secret rotation, on-call first response	Tested by following them literally
D5	At least 5 Architecture Decision Records (ADRs)	Each lists options considered and trade-offs
D6	Exported dashboards and alert rules (as code)	Recreated automatically on a fresh install
D7	Load-test scripts and a results report (before/after scaling and caching)	Includes graphs and interpretation
D8	DR drill report with measured RTO/RPO	Timestamps, what went wrong, what you changed
D9	Three post-mortems from Stage 11 incidents	Blameless format, root cause, action items
D10	Security report: hardening checklist, scan results, threat model summary	Evidence captured from outside the server
D11	Cost report	Actual monthly cost and how it would scale
D12	Learning log and a 10-minute demo video or blog post	Explains the why, not just the what

16. Evaluation Criteria

Scored out of 100, plus up to 20 bonus points. A working application with no evidence of understanding cannot score above 50.

Area	Weight	What earns full marks
Foundation & Linux administration	6	Clean, hardened, reproducible servers; sensible users, permissions and services
Git workflow & repository quality	5	Protected branches, meaningful commits, PR reviews, readable structure
Containerisation	9	Small, secure, multi-stage images; correct networks and volumes; reproducible builds
Production deployment (proxy, TLS, DNS, config)	9	A-grade TLS, correct proxying, clean separation of config and secrets

Area	Weight	What earns full marks
CI/CD	12	Fast, reliable pipeline; build-once-promote; automatic rollback; approvals
IaC & configuration management	10	Rebuild from zero is one or two commands; idempotent; no drift
Observability	11	Actionable alerts, useful dashboards, correlated logs, SLO view
Security	12	Defence in depth, least privilege, scanning, proven from outside
Reliability, backup & DR	12	Measured RTO/RPO within target; tested restores; zero-downtime deploys
Scaling & performance	6	Evidence-based limits, caching, pooling, capacity plan
Kubernetes	4	Production-style manifests: probes, resources, secrets, rolling updates
Incident response & documentation	4	Calm diagnosis, good post-mortems, runbooks others can follow
Total	100	

17. Bonus Challenges

Each bonus is worth 2–4 points (maximum 20). Pick the ones that match the job you want next.

Bonus	Description	Pts
GitOps	Kubernetes deploys driven by Argo CD or Flux from a config repo; no <code>kubectl apply</code> from CI	4
Progressive delivery	Canary or blue/green deploys with automatic promotion based on metrics	4
Distributed tracing	OpenTelemetry traces from proxy → API → DB → worker, viewable in Tempo or Jaeger	3
Supply-chain security	SBOM generation, image signing (cosign) and signature verification before deploy	3
Database replication	A streaming replica; documented promotion procedure; tested failover	4
Policy as code	Kyverno/OPA policies that reject non-compliant workloads (root users, no limits, <code>latest</code> tags)	3
Secrets manager	HashiCorp Vault or External Secrets Operator with dynamic DB credentials	3
Autoscaling	Kubernetes HPA on a custom metric (e.g. queue depth or requests per second)	3
Chaos engineering	Scheduled, automated fault injection with a hypothesis and results	3
Remote state	Terraform/OpenTofu remote state with locking and a plan-on-PR workflow	2
Status page	Public status page fed by your uptime checks	2

Bonus	Description	Pts
ChatOps	Deploy/rollback triggered and reported through a chat bot	2
Cost dashboard	Automated daily cost + resource-efficiency report	2

PART II — THE STAGES

Each stage is written as a ticket from your manager, followed by requirements, constraints, experiments, checkpoints and questions. Hints are graduated: read the first one, try again, and only then read further. Commands are intentionally absent; the hints tell you **where to look**, not what to type.

CONSTRAINT — The golden rule of every stage

At the end of each stage, destroy what you built and recreate it from the repository. If you can't, the stage is not done. This habit is the single most important thing that separates a DevOps engineer from someone who "set up a server once".

Stage 1 — Foundation

Level	Suggested time	Builds on	Primary skills
Beginner	10–14 h	—	Linux administration, users & permissions, SSH, systemd, networking basics, Git workflow, app setup

The ticket

NOTE — From: Amara Osei (CTO) — "Week 1: get the basics right"

Welcome aboard. Before anything fancy, I need a clean foundation. Set up our Git repository properly so the developers stop pushing to main, get the application running locally, and prepare a fresh server we can trust.

Also, please look at the old box description and tell me honestly how bad it is. I need a risk list I can show the board.

Objective

Establish a professional Git workflow, run the application locally without containers, and prepare a hardened-enough Linux server that you understand completely: users, SSH, packages, services, networking and storage.

Requirements

1. Create the repository with a documented branching strategy (trunk-based with short-lived branches is recommended), branch protection on **main**, a PR template, a **CODEOWNERS** file and a **.gitignore** that prevents secrets and build output from being committed.
2. Write or obtain the TrackPoint application honouring the contract in Section 3. It must run locally against a locally installed PostgreSQL and Redis **without Docker** so you understand what the containers will later replace.
3. Write a **risk register** for the legacy box (Section 4): at least 12 risks, each with likelihood, impact, and the stage in which you will fix it.
4. Provision a server (VPS or local VM) running a supported LTS release. Create a personal admin user, set up SSH key authentication, disable password and root SSH login, set the hostname, timezone (UTC) and time synchronisation.
5. Run the API on the server as a **systemd service** under a dedicated, unprivileged system user, with automatic restart and logs in the journal. (You will throw this away in Stage 2; the point is to understand what a process supervisor does.)
6. Document the server: open ports, running services, disk layout, memory, users with shell access. Produce it using commands, not guesses.
7. Write a **Makefile** or task runner with targets for the common local operations (run, test, lint).

Constraints

CONSTRAINT

No Docker in this stage.

Do not lock yourself out: always keep a second SSH session open when changing SSH or firewall configuration.

The rewritten history problem: the legacy `.env` in Git history must be identified. You will clean it in Stage 7; for now, list what it contained in the risk register.

Break it on purpose

Do these experiments deliberately. Write down what you expected, what actually happened, and why. The gap between the two is where the learning is.

- Kill the API process with different signals (`TERM`, `KILL`, `INT`). What does systemd do in each case? What does the app log?
- Make the service user unable to read the app directory. What error do you get, and where?
- Fill a test partition or a large file on disk to 100% and observe what breaks first.
- Start the app on a port already in use. Find which process owns the port.
- Create a merge conflict between two branches and resolve it; then practise `rebase` versus `merge` and explain the difference in history.

Checkpoints

Do not move to the next stage until every box is ticked **and** you could demonstrate it live.

- ☐ Pushing directly to `main` is rejected; PRs require a passing check.
- ☐ App runs locally, tests pass, `/healthz` and `/readyz` behave correctly when PostgreSQL is stopped.
- ☐ SSH as root fails; SSH with a password fails; SSH with your key as your user works.
- ☐ The API survives a reboot as a systemd service without you logging in.
- ☐ Risk register committed in `docs/`.
- ☐ Server inventory document committed.

Expected outcome

A repository that looks like a real team's repository, an application you can run and test, and a server whose every open port and running service you can explain.

Prove you understand

Answer these in writing in your `docs/learning-log.md`. If you cannot answer one without searching, you have not finished the stage.

1. What is the difference between a liveness check and a readiness check, and why would conflating them cause an outage?
2. Why should the application not run as root, even on a server you own?
3. What happens, step by step, when you run `ssh user@host`? Where is the host key stored and what is it protecting you from?
4. What does systemd do with `Restart=on-failure` that `nohup` does not? What are the limits of this approach?

5. In your branching strategy, how does a hotfix reach production?
6. Which of the legacy risks would you fix first if you had only one day, and why?

Hints (read only when stuck)

HINT — Hint 1 — Getting SSH right

Two config files matter: the server's SSH daemon configuration and your local client configuration. Look up the directives that control password authentication, root login and key authentication. Test the syntax of the daemon config before restarting it.

HINT — Hint 2 — systemd service

Read the man pages for `systemd.service` and `systemd.exec`. Look for directives controlling user, working directory, environment files, restart policy and the hardening options (`ProtectSystem`, `NoNewPrivileges`, ...).

HINT — Hint 3 — Inventory

Look at `ss`, `systemctl list-units`, `lsblk`, `df`, `free`, `/etc/passwd` and `getent`. Ask: which ports listen on all interfaces versus localhost?

Troubleshooting clues

If you see...	Look at...
Permission denied (publickey)	Permissions on <code>~/.ssh</code> and <code>authorized_keys</code> , file ownership, and the server's auth log
Service starts then immediately stops	<code>systemctl status</code> and <code>journalctl -u</code> output; working directory and environment
App works locally but not on the server	Which interface it binds to (127.0.0.1 vs 0.0.0.0), and environment variables
Address already in use	Which process owns the port and whether an old instance is still running

Study resources

- The Linux Command Line (William Shotts), free online edition
- DigitalOcean community tutorials: initial server setup, SSH essentials
- `man sshd_config`, `man systemd.service`, `man systemd.exec`
- Pro Git book (git-scm.com/book), chapters 3 and 5
- The Twelve-Factor App (12factor.net)
- Atlassian Git tutorials: comparing workflows

Stage 2 — Containerization

Level	Suggested time	Builds on	Primary skills
Beginner → Intermediate	12–16 h	Stage 1	Dockerfiles, multi-stage builds, image layers, volumes, networks, Docker Compose, health checks

The ticket

NOTE — From: Amara Osei (CTO) — "It works on my machine is banned"

Our developers keep saying it works on their machines. I want every component in a container and the whole stack to start with one command, the same way on a laptop and on a server. Keep the images small; our server doesn't have much disk.

Objective

Containerise every component with production-quality Dockerfiles and orchestrate the full stack with Docker Compose, using correct networking, persistent storage and health checks.

Requirements

1. Write multi-stage Dockerfiles for `api`, `worker` and `web`. Final images contain only what is needed at runtime and run as a non-root user.
2. Target sizes: API/worker image under 200 MB (under 100 MB is excellent); web image under 50 MB. Record the sizes and what you did to reduce them.
3. Order Dockerfile instructions for maximum layer-cache reuse and use a `.dockerignore`. Measure rebuild time after changing one source file.
4. Write a Compose file with services for web, api, worker, postgres, redis and a one-shot `migrate` service.
5. Use **at least two user-defined networks** so that the database is not on the same network as the public-facing proxy/web tier. Draw the network diagram.
6. Use named volumes for PostgreSQL data. Prove data survives `docker compose down` and container recreation, and know which command would destroy it.
7. Define health checks for every long-running service and make dependent services wait for **healthy**, not just **started**.
8. Support a local development override (hot reload, debug ports) without changing the production Compose file.
9. All configuration via environment variables with a committed `.env.example` and an ignored `.env`.

Constraints

CONSTRAINT

No `latest` tags on base images: pin versions (bonus: pin by digest).

No database port published to the host in the production Compose configuration.

The `migrate` service must run to completion before the API starts in a fresh environment.

Break it on purpose

Do these experiments deliberately. Write down what you expected, what actually happened, and why. The gap between the two is where the learning is.

- Remove the named volume declaration and restart. Where did the data go?
- Put the API on a network the database is not on. What error does the API show? What does `/readyz` say?
- Set a memory limit of 32 MB on the API. What does `docker inspect` report about the container's last exit?
- Change one line of source code and rebuild with a badly ordered Dockerfile, then with a well-ordered one. Compare times.
- Run the container as root, then as non-root, and try to write to a directory owned by root. Fix it properly (not by going back to root).

Checkpoints

Do not move to the next stage until every box is ticked **and** you could demonstrate it live.

- ☐ `docker compose up` from a clean clone brings up a fully working stack.
- ☐ Images meet size targets and run as non-root (verify with the actual process UID inside the container).
- ☐ PostgreSQL is not reachable from the host network in production mode.
- ☐ Data survives container recreation; you can explain exactly which command deletes it.
- ☐ Stopping PostgreSQL makes the API report not-ready; restarting it recovers without restarting the API.
- ☐ Network diagram committed.

Expected outcome

A reproducible, one-command stack where you understand how containers find each other, where data lives, and why each image is as small as it is.

Prove you understand

Answer these in writing in your `docs/learning-log.md`. If you cannot answer one without searching, you have not finished the stage.

1. Explain how container name resolution works on a user-defined bridge network, and why it doesn't work on the default bridge.
2. What is the difference between an image layer, a container's writable layer, a bind mount and a named volume?
3. Why does `depends_on` without a health condition not solve start-up ordering?
4. What is PID 1 inside a container and why does it matter for signal handling and zombie processes?
5. What do `EXPOSE` and `ports:` actually do? Which one opens a firewall hole?
6. Why is a multi-stage build more secure, not just smaller?

Hints (read only when stuck)

HINT — Hint 1 — Image size

Inspect the image layer by layer (the `docker history` command, or a tool called `dive`). The big wins usually come from the base image choice and from not shipping build toolchains.

HINT — Hint 2 — Start-up order

Look at the long syntax of `depends_on` in the Compose specification and the conditions it supports. One of them is designed exactly for one-shot jobs like migrations.

HINT — Hint 3 — Signals

If your container takes 10 seconds to stop, your app probably isn't receiving `SIGTERM`. Compare the shell form and exec form of `CMD/ENTRYPOINT`.

Troubleshooting clues

If you see...	Look at...
<code>connection refused</code> to <code>localhost:5432</code> from the API	What <code>localhost</code> means inside a container
Container exits with code 137	Memory limits and the OOM killer
Permission denied writing files in container	File ownership in the image versus the runtime user; volume ownership
Changes to code don't appear	Build cache, which image tag Compose is running, bind mounts in the override file
Container "healthy" but app broken	What your health check actually tests

Study resources

- Docker docs: Dockerfile best practices, multi-stage builds, networking overview
- Compose Specification (compose-spec.io), especially `depends_on`, `healthcheck`, `profiles`
- `dive` (image layer explorer) and `hadolint` (Dockerfile linter)
- Docker security cheat sheet (OWASP)

Stage 3 — Production Deployment

Level	Suggested time	Builds on	Primary skills
Intermediate	12–16 h	Stages 1–2	Reverse proxy, DNS, HTTPS/ACME, environment configuration, PostgreSQL operations

The ticket

NOTE — From: Jonas Keller (Operations) — "Clients need a real URL"

Our pilot clients need to hit `https://track.<our-domain>` for tracking and `https://api.<our-domain>` for integrations. Staging should be separate so we can test without touching real data. And please, the browser must not show any security warnings.

Objective

Put the stack on the internet properly: a domain with correct DNS records, a reverse proxy that terminates TLS with automatically renewed certificates, separate staging and production configurations, and a correctly configured PostgreSQL.

Requirements

1. Register or reuse a domain (a cheap one is fine) and create DNS records for `track`, `api`, `staging-track` and `staging-api` subdomains. Choose TTLs deliberately and write down why.
2. Deploy a reverse proxy (Nginx is recommended for learning; Traefik or Caddy are acceptable if you can explain the ACME flow they automate) as the **only** container with published ports.
3. Obtain certificates via ACME (Let's Encrypt). Renewal must be automatic and you must **prove** it works using a dry run or staging CA.
4. Redirect HTTP to HTTPS, enable HSTS, set security headers, gzip/brotli compression and sensible timeouts and body-size limits.
5. Forward the correct client IP and protocol headers to the API; the API logs the real client IP, not the proxy's.
6. Route `/metrics` so it is **not** reachable from the internet.
7. Separate staging and production: different Compose project names, databases, secrets and domains. They may share a server at this stage.
8. Configure PostgreSQL deliberately: memory settings suited to your server size, connection limit, and logging of slow queries (> 250 ms).
9. Create the database roles required by SEC-06 (migration owner, application user, read-only user) and use them.

Constraints

CONSTRAINT

Let's Encrypt has rate limits. Use the staging CA while experimenting; you will be blocked for days if you hit production limits.

No self-signed certificates in production. (Allowed only in the no-budget local variant, documented.)

Break it on purpose

Do these experiments deliberately. Write down what you expected, what actually happened, and why. The gap between the two is where the learning is.

- Point a DNS record at the wrong IP, fix it, and measure how long different resolvers take to notice. Relate it to TTL.
- Make the proxy send traffic to an API container that is stopped. What status code does the client see, and what does the proxy log say?
- Set a proxy read timeout of 1 s and call `/api/reports/daily`. Identify which side closed the connection.
- Block port 80 in the firewall and attempt a certificate renewal dry run. Understand why HTTP-01 challenges need port 80.

Checkpoints

Do not move to the next stage until every box is ticked **and** you could demonstrate it live.

- ☐ All four hostnames serve over valid HTTPS; HTTP redirects to HTTPS.
- ☐ External TLS test (e.g. SSL Labs) grades A or better; screenshot committed.
- ☐ Certificate renewal dry-run succeeds.
- ☐ `/metrics` returns 403/404 from the internet but is reachable from the internal network.
- ☐ API logs show real client IPs.
- ☐ The application DB user cannot `DROP TABLE` (prove it).
- ☐ Staging and production are fully isolated; wiping staging cannot affect production.

Expected outcome

Real users can reach TrackPoint securely at a proper domain, and you can explain every hop of a request from the browser's DNS lookup to the database query.

Prove you understand

Answer these in writing in your `docs/learning-log.md`. If you cannot answer one without searching, you have not finished the stage.

1. Walk through what happens between typing `https://track.<domain>` and the first byte of HTML: DNS, TCP, TLS handshake, proxy, upstream.
2. What is the difference between HTTP-01 and DNS-01 ACME challenges? When would you need DNS-01?
3. What is HSTS, and what could go wrong if you enable `preload` too early?
4. Why should `X-Forwarded-For` only be trusted from known proxies?
5. What is the difference between an A record, a CNAME and an ALIAS/ANAME record? What is the risk of a very long TTL during a migration?
6. Why does PostgreSQL have a connection limit, and what happens to memory as connections grow?

Hints (read only when stuck)

HINT — Hint 1 — Certificates in containers

Decide where certificates live (a volume shared between the proxy and the ACME client) and how the proxy picks up a renewed certificate without downtime. Look up the difference between a reload and a restart for your proxy.

HINT — Hint 2 — Real client IP

Search for the real-IP module/directive in your proxy and the equivalent "trust proxy" setting in your API framework. Both sides must agree.

HINT — Hint 3 — PostgreSQL tuning

Start from a tuning calculator such as PGTune using your actual RAM, then reduce the values because other containers share the server. Justify each change.

Troubleshooting clues

If you see...	Look at...
502 Bad Gateway	Upstream name/port, whether proxy and API share a network, API readiness
504 Gateway Timeout	Proxy timeouts versus slow queries
Certificate is for the wrong name	Server blocks/router rules and SNI; which certificate is served as default
Redirect loop	Proxy forwarding the scheme header; app thinks the request is HTTP
ACME challenge fails	DNS propagation, port 80 reachability, challenge path routing

Study resources

- Nginx docs: reverse proxy, [ngx_http_realip_module](#); or Traefik / Caddy docs
- Let's Encrypt docs: how it works, challenge types, rate limits
- Mozilla SSL Configuration Generator; Mozilla Observatory
- Cloudflare Learning Center: DNS records and TTL
- PostgreSQL docs: server configuration, roles and privileges, [GRANT](#)

Stage 4 — CI/CD

Level	Suggested time	Builds on	Primary skills
Intermediate	14–18 h	Stages 1–3	Pipelines, automated testing, container registries, artifact promotion, deploy automation, rollback

The ticket

NOTE — From: Amara Osei (CTO) — "No more scp"

Last Friday a developer deployed an untested change by copying files to the server and broke client label printing for three hours. From now on, nothing reaches production without passing through a pipeline. I want developers to be able to ship on their own, but safely. And if a deploy goes wrong, I want it undone in minutes, not hours.

Objective

Build a CI/CD pipeline that tests every change, builds each image exactly once, promotes the same artifact from staging to production with an approval gate, and rolls back automatically when a deploy fails.

Requirements

1. Implement CI-01 to CI-09 (Section 12) using GitHub Actions or GitLab CI (recommended; free tiers are sufficient).
2. Integration tests run against a real PostgreSQL service container inside the pipeline.
3. Push images to a registry (GHCR, GitLab Registry or Docker Hub) tagged with the full commit SHA. Optionally add a human-readable semver tag on releases.
4. Design how the pipeline reaches the server: SSH with a restricted deploy key, a pull-based agent, or a self-hosted runner. Write an ADR comparing at least two options, including their security implications.
5. The deploy step: pull the new image, run migrations with the migration role, update services, wait for readiness, run the smoke test.
6. If the smoke test fails, the pipeline automatically redeploys the previous SHA and fails loudly.
7. Record each deployment (DEP-07).
8. Make `/api/version` show the deployed SHA so anyone can verify what is live.

Constraints

CONSTRAINT

The production server never runs `docker build`.

Secrets live only in the CI secret store or your chosen secret mechanism, never in workflow files.

Production deploys require manual approval via the platform's environment protection feature.

Break it on purpose

Do these experiments deliberately. Write down what you expected, what actually happened, and why. The gap between the two is where the learning is.

- Merge a commit with a failing unit test. Confirm nothing is built or deployed.
- Deploy a version whose API crashes on start (e.g. a required environment variable is missing). Watch the automatic rollback happen and time it.
- Revoke the registry token used by the pipeline. Read the resulting error and fix it properly.
- Poison the dependency cache (change a lock file without changing the cache key). Observe what happens.
- Deploy while running a continuous request loop against the API. Count failed requests. (You will fix this in Stage 8; for now, measure the baseline.)

Checkpoints

Do not move to the next stage until every box is ticked **and** you could demonstrate it live.

- ☐ A PR shows lint, test, scan and build results; merge is blocked on failure.
- ☐ Merge to `main` → staging deploy with no human action.
- ☐ Approval → production deploy of the identical image digest used in staging (prove it by comparing digests).
- ☐ A broken release is rolled back automatically in under 5 minutes.
- ☐ PR pipeline runs in under 10 minutes.
- ☐ Deploy history is visible.

Expected outcome

Developers ship through pull requests; you no longer touch production by hand; bad releases undo themselves.

Prove you understand

Answer these in writing in your `docs/learning-log.md`. If you cannot answer one without searching, you have not finished the stage.

1. Why "build once, deploy many"? What specifically could differ if you rebuilt the image for production?
2. What is the difference between an image tag and an image digest? Which one should a deploy reference, and why?
3. Continuous delivery versus continuous deployment: which did you implement, and why is it right for Harborline now?
4. What can an attacker do with your deploy credential? How did you limit that blast radius?
5. Why must migrations run before the new code starts, and what makes a migration unsafe to roll back?
6. How do you prevent two pipelines from deploying at the same time?

Hints (read only when stuck)

HINT — Hint 1 — Promotion

Staging and production jobs should consume the image reference produced by the build job as an output, not rebuild it. Look at job outputs and environments in your CI platform.

HINT — Hint 2 — Restricted SSH

OpenSSH can restrict what a specific key is allowed to do from the `authorized_keys` file itself. Research the options available per key.

HINT — Hint 3 — Rollback

Rollback is only easy if the previous version is always known. Where does your pipeline record "the version currently live" before it changes it?

HINT — Hint 4 — Concurrency

Most CI platforms have a concurrency or resource-group feature. Look it up.

Troubleshooting clues

If you see...	Look at...
Tests pass locally but fail in CI	Service container readiness, hostnames, timezone, missing env vars
<code>denied: permission</code> pushing images	Token scopes and package/repository permissions
Deploy "succeeds" but old version still live	Which tag the server pulled; image pull policy; whether containers were recreated
Pipeline suddenly slow	Cache hit/miss in logs; cache key design
Host key verification failed	Known-hosts handling in the pipeline (and why disabling the check is wrong)

Study resources

- GitHub Actions docs: workflows, environments, reusable workflows, concurrency, OIDC
- GitLab CI docs (if using GitLab)
- Docker docs: build cache in CI, `docker buildx`
- Martin Fowler: Continuous Integration; Jez Humble & David Farley, Continuous Delivery
- Trivy or Gripe documentation; gitleaks documentation

Stage 5 — Infrastructure as Code

Level	Suggested time	Builds on	Primary skills
Intermediate → Advanced	14–18 h	Stages 1–4	Terraform/OpenTofu, state management, Ansible, idempotency, drift, environment parity

The ticket

NOTE — From: Amara Osei (CTO) — "What if you get hit by a bus?"

If our only server disappeared tonight, how long would it take to rebuild? If the answer involves your memory, that's a risk I can't accept. I want the servers, firewall, DNS and everything on them described in code.

Objective

Make the entire platform reproducible: servers, networking, firewall and DNS provisioned with Terraform/OpenTofu, and every host configured with Ansible, so a fresh environment can be created from an empty cloud account.

Requirements

1. Use Terraform or OpenTofu to provision: servers (staging and production as separate hosts if budget allows), SSH keys, cloud firewall rules, DNS records, and object storage for backups (if your provider supports it).
2. Structure IaC with reusable modules and per-environment configuration. No copy-pasted resource blocks between environments.
3. Store state safely (remote backend with locking is ideal). Never commit state files; explain why.
4. Use Ansible (or an equivalent) to configure hosts: users, SSH hardening, packages, Docker installation, firewall, unattended upgrades, log rotation, application directory, Compose files and systemd units.
5. Organise Ansible into roles with variables per environment; secrets encrypted (Ansible Vault or SOPS).
6. Everything is idempotent: running the playbook twice in a row reports zero changes the second time.
7. Provide a single documented entry point (e.g. `make infra-up ENV=staging`) that goes from nothing to a working environment.
8. Replace your hand-built Stage 1 server with a code-built one. Update DNS through code.

Constraints

CONSTRAINT

No manual changes in the cloud console. If you must make one during an emergency, capture it in code the same day (and log it).

Terraform does provisioning; Ansible does configuration. Document where you drew the line and why (e.g. cloud-init versus Ansible).

Break it on purpose

Do these experiments deliberately. Write down what you expected, what actually happened, and why. The gap between the two is where the learning is.

- Change a firewall rule by hand in the console, then run a plan. Read the drift report.
- Delete the local state file (on a throwaway environment!) and see what Terraform now thinks. Learn how import works.
- Write a deliberately non-idempotent Ansible task (e.g. appending a line with a shell command). Run it three times and inspect the result. Rewrite it properly.
- Destroy staging completely and rebuild it. Time it.

Checkpoints

Do not move to the next stage until every box is ticked **and** you could demonstrate it live.

- ☐ `plan` against the live environment shows no changes.
- ☐ Second Ansible run: `changed=0`.
- ☐ Staging destroyed and rebuilt from code; time recorded.
- ☐ No secrets or state files in the repository (verify with a secret scanner).
- ☐ DNS records managed by code.

Expected outcome

Your infrastructure lives in Git. Rebuilding is a command, not a memory exercise.

Prove you understand

Answer these in writing in your `docs/learning-log.md`. If you cannot answer one without searching, you have not finished the stage.

1. What is Terraform state, what does it contain that makes it sensitive, and what happens if two people apply at the same time without locking?
2. Declarative versus imperative: where does Ansible sit, and what makes a task idempotent?
3. What is configuration drift, how does it happen in real companies, and how would you detect it continuously?
4. Where did you draw the line between Terraform, cloud-init and Ansible, and what would change your mind?
5. Immutable infrastructure versus mutable configuration management: what are the trade-offs for a team of four?

Hints (read only when stuck)

HINT — Hint 1 — Connecting Terraform and Ansible

Terraform knows the new server's IP; Ansible needs it. Look at Terraform outputs and Ansible dynamic inventory or generated inventory files.

HINT — Hint 2 — Idempotency

Prefer purpose-built Ansible modules over `shell/command`. When you must use them, look up the `creates`, `changed_when` and `when` keywords.

HINT — Hint 3 — Remote state on a budget

Many S3-compatible object stores work as a Terraform backend. Some CI platforms also offer managed state. Research locking support for each.

Troubleshooting clues

If you see...	Look at...
Terraform wants to recreate a server you only tweaked	Which attribute forces replacement in the plan output
Ansible hangs on first connection	Host key checking, the Python interpreter on the target, SSH user
Playbook works once, fails on re-run	Tasks that aren't idempotent; handlers; ordering
Firewall rules applied but traffic still blocked	Cloud firewall vs host firewall vs Docker's own iptables rules

Study resources

- Terraform / OpenTofu docs: state, backends, modules, `import`
- Terraform: Up & Running (Yevgeniy Brikman)
- Ansible docs: roles, idempotency, Vault; Jeff Geerling, Ansible for DevOps
- Your cloud provider's Terraform provider documentation
- Research: how Docker interacts with UFW/iptables (a classic trap)

Stage 6 — Observability

Level	Suggested time	Builds on	Primary skills
Intermediate → Advanced	14–18 h	Stages 1–5	Metrics, logs, dashboards, alerting, health checks, SLOs, correlation

The ticket

NOTE — From: Jonas Keller (Operations) — "I don't want to hear about outages from customers again"

During the last outage our first signal was an angry tweet. I want us to know about problems before our clients do, and when something breaks I want to see why, not just that it broke.

Objective

Build a monitoring, logging and alerting system that gives early, actionable warning of problems and lets you move from "something is wrong" to "this is the cause" in minutes.

Requirements

1. Deploy a metrics stack (Prometheus + Grafana recommended) with exporters for hosts, containers, PostgreSQL, Redis and the proxy. Scrape the API's `/metrics` over the internal network.
2. Deploy log aggregation (Loki + Promtail/Alloy recommended, or an alternative) collecting all container logs with labels for service and environment.
3. Build dashboards satisfying MON-02, provisioned from files in Git, not clicked together by hand.
4. Define alert rules as code for every item in MON-05, routed through Alertmanager (or equivalent) to a real notification channel.
5. Add an external uptime check from a free external service. Explain what it detects that your internal monitoring cannot.
6. Write a runbook for each alert (MON-06) and link it from the alert annotation.
7. Define SLIs and SLOs for availability and latency; build an error-budget panel.
8. Measure and document the observability stack's own resource usage and retention settings; make sure it fits the budget.
9. Demonstrate correlation: from a latency spike on a dashboard, find the matching request IDs in logs, and the slow query in PostgreSQL logs.

Constraints

CONSTRAINT

Monitoring UIs are never publicly exposed without authentication (SEC-11). Preferably reach them via SSH tunnel or VPN.

Log retention must not endanger the disk. Calculate expected daily log volume.

Monitoring that runs on the same host it monitors cannot tell you the host is dead. Address this.

Break it on purpose

Do these experiments deliberately. Write down what you expected, what actually happened, and why. The gap between the two is where the learning is.

- Stop the API. How many seconds until you are notified? Which notification arrives first, internal or external?
- Generate a burst of 500 errors. Does your alert fire? Does it resolve by itself afterwards?
- Fill the disk to 85% with a junk file. Confirm the alert, then clean up.
- Make the API log a non-JSON line. What happens to it in your log pipeline?
- Set an alert threshold too sensitive, let it flap for an hour, and then tune it. Document the before/after.

Checkpoints

Do not move to the next stage until every box is ticked **and** you could demonstrate it live.

- ☐ All targets show UP in Prometheus (or equivalent).
- ☐ Dashboards and alert rules come back automatically after destroying and recreating the monitoring stack.
- ☐ Every MON-05 alert has been triggered at least once in a test and delivered to your phone/inbox.
- ☐ External uptime check configured and alerting.
- ☐ SLO/error-budget panel exists.
- ☐ Correlation demo recorded (screenshots or short video).

Expected outcome

You get paged for real problems and not for noise, and a dashboard + logs let you narrow down causes quickly.

Prove you understand

Answer these in writing in your `docs/learning-log.md`. If you cannot answer one without searching, you have not finished the stage.

1. Explain the RED and USE methods. Which dashboards did you build from each?
2. Why are latency percentiles (p95/p99) more useful than averages? What is a histogram bucket and how does its choice affect accuracy?
3. What is the difference between monitoring and observability?
4. What makes an alert actionable? Give an example of an alert you decided **not** to create and why.
5. Why can high-cardinality labels (e.g. user ID) take down a Prometheus server?
6. How much downtime does a 99.5% monthly SLO allow, and what should the team do when the error budget is spent?

Hints (read only when stuck)

HINT — Hint 1 — Provisioning Grafana

Grafana can load data sources and dashboards from files at start-up. Search for "Grafana provisioning". Build dashboards in the UI, export the JSON, commit it.

HINT — Hint 2 — Symptom vs cause alerts

Alert on user-visible symptoms (errors, latency, unavailability) and on predictable future failures (disk filling, certificate expiry). Treat most cause-level signals (high CPU) as dashboard material, not pages.

HINT — Hint 3 — Backup freshness

Have the backup job publish a "last success" timestamp metric (the Pushgateway or the node exporter textfile collector are two ways). Alert when "now minus last success" is too large.

Troubleshooting clues

If you see...	Look at...
Target DOWN in Prometheus	Network membership, target hostname/port, whether <code>/metrics</code> is bound on the internal interface
Alerts in Prometheus but no notifications	Alertmanager routing tree, receiver config, <code>for:</code> duration
Logs missing for one container	Docker logging driver, labels, agent permissions to the Docker socket/log files
Grafana shows "No data"	Time range, query labels, data source URL from inside the Grafana container
Prometheus memory keeps growing	Cardinality, scrape interval, retention

Study resources

- Google SRE Book: chapters on monitoring distributed systems and SLOs (free online)
- Prometheus docs: instrumentation, histograms, alerting rules; Alertmanager routing
- Grafana docs: provisioning; Grafana Loki docs
- Brendan Gregg: the USE method; Tom Wilkie: the RED method
- Rob Ewaschuk: My Philosophy on Alerting

Stage 7 — Security

Level	Suggested time	Builds on	Primary skills
Advanced	12–16 h	Stages 1–6	Hardening, secrets management, least privilege, container security, scanning, threat modelling

The ticket

NOTE — From: Legal advisor, cc Amara Osei — "Pilot contract, clause 9: security"

The pilot contracts require us to demonstrate "appropriate technical measures" to protect customer personal data. Before go-live I need a short security report with evidence: who can access what, how secrets are managed, how you'd know about a breach.

Also, the old repository still contains credentials in its history. Please deal with that.

Objective

Apply defence in depth across host, network, containers, secrets, CI/CD and the database, and produce evidence (from outside the system) that SEC-01 to SEC-14 are met.

Requirements

1. Write a lightweight **threat model**: assets, trust boundaries, entry points, top 10 threats (STRIDE is a good framework) and your mitigations.
2. Handle the leaked legacy `.env`: rotate every credential, then decide whether to rewrite Git history. Document why rotation is mandatory even if history is rewritten.
3. Implement SEC-05 with your chosen secret mechanism; secrets are encrypted at rest in Git (if stored there) and decrypted only at deploy time.
4. Harden containers: non-root, dropped capabilities, `no-new-privileges`, read-only root filesystem with explicit writable tmp volumes where needed, resource limits.
5. Harden hosts: SSH (SEC-01), firewall (SEC-02, including the Docker/firewall interaction), brute-force protection, unattended upgrades (SEC-12), and a CIS-style audit using a tool such as Lynis. Fix at least the top 10 findings or justify why not.
6. Enforce image and dependency scanning (SEC-08) and secret scanning (SEC-09) in CI; add a nightly re-scan of deployed images.
7. Add rate limiting (SEC-14) and verify it with a test.
8. Restrict monitoring and admin surfaces (SEC-11) with a VPN (e.g. WireGuard or Tailscale), SSH tunnels, or authentication plus an IP allowlist.
9. Run an external port scan and TLS test against your production IP and domain; commit the results.

Constraints

CONSTRAINT

Hardening must not break deploys, monitoring or backups. Re-run your full pipeline and restore test afterwards.

Do not scan infrastructure you do not own.

Break it on purpose

Do these experiments deliberately. Write down what you expected, what actually happened, and why. The gap between the two is where the learning is.

- From outside, try to connect to PostgreSQL, Redis, Prometheus and Grafana ports. Every attempt should fail. Then deliberately publish the Redis port in Compose and see whether your host firewall actually blocks it (this surprises most people).
- Commit a fake AWS-style key on a branch. Confirm CI blocks it.
- Build an image from an old base image with known CVEs and watch the pipeline fail.
- Try to write to `/` inside a read-only container. Find the paths your app legitimately needs to write to.
- Hammer the login/API endpoint and confirm rate limiting responds with 429.

Checkpoints

Do not move to the next stage until every box is ticked **and** you could demonstrate it live.

- ☐ External scan shows only 80/443 (and SSH if not tunnelled/restricted).
- ☐ All legacy credentials rotated; rotation logged.
- ☐ CI blocks secrets, critical CVEs and Dockerfile anti-patterns.
- ☐ Containers verified non-root with dropped capabilities.
- ☐ Lynis (or equivalent) hardening index improved; before/after committed.
- ☐ Security report and threat model committed.

Expected outcome

A defensible security posture with evidence, not a list of intentions.

Prove you understand

Answer these in writing in your `docs/learning-log.md`. If you cannot answer one without searching, you have not finished the stage.

1. Why is rotating a leaked secret mandatory even after you rewrite Git history?
2. Explain why Docker-published ports can bypass UFW, and how you prevented it.
3. What does "least privilege" mean concretely for: the SSH user, the deploy key, the DB application role and the containers?
4. What is the difference between a vulnerability in the base image and one in an application dependency, and how is each fixed?
5. Mounting the Docker socket into a container (e.g. for a log collector) is common. What does it give that container, and how would you reduce the risk?
6. If an attacker gained a shell inside the API container, what could they reach? Walk through your layers of defence.

Hints (read only when stuck)

HINT — Hint 1 — Secrets choice

For a small team, compare: CI platform secrets written to a root-only env file at deploy time; SOPS + age files in Git; Docker/Compose secrets mounted as files; Vault. Consider rotation, auditability and what happens during a DR rebuild.

HINT — Hint 2 — Docker and the firewall

Research the **DOCKER-USER** chain and binding published ports to **127.0.0.1** explicitly.

HINT — Hint 3 — History rewrite

Look at **git filter-repo** and BFG Repo-Cleaner, and at what happens to existing clones and forks after a force-push.

Troubleshooting clues

If you see...	Look at...
App crashes with read-only filesystem	Which paths it writes to (temp files, caches, PID files); mount tmpfs for those only
SSH locked out after hardening	Your provider's web console / rescue mode; keep a second session open next time
Scanner reports hundreds of CVEs	Base image choice; filter by severity and fix availability; unused packages
Rate limit blocks legitimate clients	Key (per IP vs per API key), burst settings, clients behind NAT

Study resources

- OWASP Docker Security Cheat Sheet; OWASP Top 10
- CIS Benchmarks for Ubuntu and Docker (summaries); Lynis documentation
- Microsoft threat modelling / STRIDE overview
- SOPS and age documentation; HashiCorp Vault getting started
- WireGuard or Tailscale quickstart
- Trivy, Gripe, gitleaks, hadolint documentation

Stage 8 — Reliability

Level	Suggested time	Builds on	Primary skills
Advanced	14–18 h	Stages 1–7	Backups, point-in-time recovery, DR, zero-downtime deploys, safe migrations, rollback

The ticket

NOTE — From: Amara Osei (CTO) — "Prove we can survive the worst day"

The board asked me a simple question: "If the server is destroyed at 3 p.m. on a Tuesday, what happens?" I want an answer with numbers, and I want those numbers measured, not estimated.

Also, developers complain that deploys cause a few seconds of errors. Clients' scanners retry, but they log errors and our clients notice.

Objective

Guarantee the business can recover from data loss and total infrastructure loss within RPO/RTO, and make routine deployments invisible to users.

Requirements

1. Implement BAK-01 to BAK-08. For $RPO \leq 1$ h, combine regular base backups with WAL archiving, or use a tool that does both (e.g. pgBackRest, WAL-G). Explain why `pg_dump` alone cannot meet this.
2. Encrypt backups client-side and ship them off-site. Keep the decryption key somewhere that survives the loss of the server **and** of your laptop.
3. Automate a weekly restore test into a scratch database with a sanity query; export success/failure as a metric; alert on failure.
4. Perform a **point-in-time recovery** to a moment just before a simulated accidental **DELETE**.
5. Perform the full **DR drill** (BAK-07): destroy production, rebuild from code and backups, restore DNS. Measure and report RTO and RPO.
6. Implement zero/minimal-downtime deploys (DEP-03): run multiple API instances and replace them one at a time, draining connections, or use blue/green with the proxy switching upstreams. Prove it with a continuous load test during deploy.
7. Practise an expand/contract migration: rename a column across three releases without downtime.
8. Make the stack survive a host reboot with correct ordering, and survive the loss of Redis (what happens to queued jobs?).

Constraints

CONSTRAINT

Restores are tested on isolated infrastructure. Never test a restore by overwriting production.

The DR drill must be done with the runbook open, following it literally. Every deviation is a runbook bug to fix.

Break it on purpose

Do these experiments deliberately. Write down what you expected, what actually happened, and why. The gap between the two is where the learning is.

- Corrupt a backup file on purpose. Does your restore test catch it?
- Let the backup target credentials expire. How long until you know?
- Run `DELETE FROM parcels;` in staging (with no `WHERE`). Recover to the minute before.
- Deploy while `k6` runs at 50 req/s. Count the failures before and after your zero-downtime work.
- Reboot the server during business hours (staging). Time until fully healthy.

Checkpoints

Do not move to the next stage until every box is ticked **and** you could demonstrate it live.

- ☐ Off-site encrypted backups with automatic retention.
- ☐ Weekly automated restore test green, with alerting.
- ☐ PITR demonstrated in staging.
- ☐ DR drill report with measured RTO ≤ 2 h and RPO ≤ 1 h.
- ☐ Deploy under load: $< 0.1\%$ failures (target zero).
- ☐ Expand/contract migration completed with no errors.

Expected outcome

Numbers you can put in front of a board: measured recovery times, proven restores, and deploys nobody notices.

Prove you understand

Answer these in writing in your `docs/learning-log.md`. If you cannot answer one without searching, you have not finished the stage.

1. Define RPO and RTO. Which parts of your design determine each?
2. Why is a backup that has never been restored not a backup? Name three ways a backup can silently be useless.
3. What is the PostgreSQL write-ahead log and how does it enable point-in-time recovery?
4. Compare rolling, blue/green and canary deployments: what does each cost in resources, complexity and risk?
5. Why can a database migration make rollback impossible, and how does expand/contract prevent that?
6. What is connection draining, and what goes wrong if the proxy and the app disagree on timeouts during a deploy?

Hints (read only when stuck)

HINT — Hint 1 — Zero-downtime with Compose

Compose alone does not do rolling updates gracefully. Think about running two sets of API containers and switching the proxy upstream (blue/green), or scaling up before scaling down. Combine this with the proxy's passive/active health checks and a graceful `SIGTERM` handler.

HINT — Hint 2 — Where the RTO goes

Time each phase of your DR drill separately: provisioning, configuration, image pull, data restore, DNS propagation. The slowest phase is usually not the one you expect.

HINT — Hint 3 — Key management

If the backup encryption key is only on the server, your off-site backups die with the server. Think about where a copy of the key must live.

Troubleshooting clues

If you see...	Look at...
WAL archive keeps growing	Archive command failures, retention settings, replication slots
Restore works but app errors	Role/ownership mismatch, missing extensions, sequences, migrations table version
A few 502s during deploy	Proxy still routing to stopping containers; readiness timing; keep-alive connections
DNS still points at old server after rebuild	TTL and resolver caching; whether the IP is static/floating

Study resources

- PostgreSQL docs: continuous archiving and PITR
- pgBackRest or WAL-G documentation; restic documentation (for file-level backups)
- Google SRE Book: data integrity chapter
- Martin Fowler: BlueGreenDeployment; "Parallel Change" (expand/contract)
- k6 documentation (load testing)

Stage 9 — Scaling

Level	Suggested time	Builds on	Primary skills
Advanced	12–16 h	Stages 1–8	Load balancing, horizontal scaling, caching, connection pooling, resource limits, capacity planning

The ticket

NOTE — From: Jonas Keller (Operations) — "Holiday peak is coming"

Last year's holiday week was triple our normal volume and the old system crawled. The daily report page took so long that the operations team gave up on it. We can't buy big servers; make better use of what we have.

Objective

Scale TrackPoint horizontally within budget, eliminate the obvious bottlenecks, and back every limit and design decision with load-test data.

Requirements

1. Establish a baseline: run the load profile from Section 8 against staging and record latency, throughput, error rate and resource usage.
2. Find the first bottleneck using your dashboards (not guesses). Fix it, re-test, and repeat at least three times. Record each iteration.
3. Run ≥ 3 API replicas behind a load balancer with health checks (SCL-01); prove stateless behaviour by killing replicas mid-test (SCL-02).
4. Add Redis caching for the daily report with a TTL and invalidation strategy; measure hit ratio.
5. Introduce connection pooling (application pool sizing and/or PgBouncer) and prove that replicas $\times$ pool size is bounded (SCL-05).
6. Set CPU/memory limits and reservations for every container based on observed usage plus headroom (SCL-06).
7. Scale the worker independently and graph queue depth versus worker count.
8. Demonstrate scale-out to a second host with a load balancer in front (SCL-07). The DB may remain on one host; document that single point of failure.
9. Write the capacity plan (SCL-08).

Constraints

CONSTRAINT

Stay within the USD 30/month budget. If scale-out needs a second host, remove something else or justify a short-lived test host.

Load tests run from outside the target host.

Break it on purpose

Do these experiments deliberately. Write down what you expected, what actually happened, and why. The gap between the two is where the learning is.

- Set the pool size so high that replicas × pool exceeds `max_connections`. Scale up and observe the failure mode.
- Set the report cache TTL to 24 h, then change the data. Is the stale result acceptable? For whom?
- Kill one API replica during a load test. Does the load balancer stop routing to it, and how fast?
- Remove CPU limits and hammer `/api/reports/daily`. What happens to `/api/parcels/{id}` latency? This is the "noisy neighbour" problem.
- Flush Redis during peak load. What is the thundering-herd effect on PostgreSQL?

Checkpoints

Do not move to the next stage until every box is ticked **and** you could demonstrate it live.

- ☐ Load-test report with at least 3 before/after iterations and graphs.
- ☐ Section 8 performance targets met (or gaps explained with data).
- ☐ Replica killed during test without significant error spike.
- ☐ Cache hit ratio visible on a dashboard.
- ☐ Connections bounded; no "too many connections" errors at peak.
- ☐ Capacity plan committed.

Expected outcome

A platform that handles 3× traffic on a small budget, and a capacity plan that says what breaks next.

Prove you understand

Answer these in writing in your `docs/learning-log.md`. If you cannot answer one without searching, you have not finished the stage.

1. Vertical versus horizontal scaling: when is each the right answer for Harborline?
2. What makes a service stateless? Which parts of TrackPoint are inherently stateful, and how are they handled?
3. Explain round-robin, least-connections and IP-hash load balancing. Which did you choose?
4. Cache invalidation is famously hard. Describe your strategy and its failure modes.
5. What is the difference between a CPU limit and a CPU request/reservation? What is CPU throttling and how do you detect it?
6. According to your capacity plan, what is the next bottleneck, and what would you do about it?

Hints (read only when stuck)

HINT — Hint 1 — Finding bottlenecks

At each load level, check in order: error rate, latency percentiles, saturation of CPU/memory/connections per component. The component that saturates first is the bottleneck; everything else is a symptom.

HINT — Hint 2 — Pooling

Research PgBouncer's pool modes and what features each breaks (e.g. prepared statements, session variables).

HINT — Hint 3 — The report endpoint

Before caching, look at the query plan. A missing index or an N+1 query pattern can be a 10× improvement for free.

Troubleshooting clues

If you see...	Look at...
Latency rises but CPU is low	Connection pool wait time, DB locks, synchronous external calls
One replica gets most traffic	Keep-alive connections, load-balancing algorithm, DNS-based balancing caching
Memory grows steadily under load	Leak versus cache growth; limits and OOM kills in metrics
<code>too many connections</code>	<code>max_connections</code> , pool sizes per replica, idle connections

Study resources

- Nginx/HAProxy docs: upstreams, health checks, load-balancing methods
- PgBouncer docs; PostgreSQL `EXPLAIN ANALYZE` docs; use-the-index-luke.com
- Redis docs: eviction policies, key expiration
- k6 docs: scenarios, thresholds; or Locust docs
- Designing Data-Intensive Applications (Martin Kleppmann), chapters 1 and 5

Stage 10 — Kubernetes

Level	Suggested time	Builds on	Primary skills
Advanced	16–22 h	Stages 1–9	k8s objects, probes, resources, config & secrets, ingress, rolling updates, stateful workloads, Helm/Kustomize

The ticket

NOTE — From: Amara Osei (CTO) — "Pilot: are we ready for Kubernetes?"

Investors keep asking if we're "on Kubernetes". I don't want to move for fashion. Run a pilot: move TrackPoint to a small cluster using the same images, implement it the way a serious company would, and then tell me honestly whether it's worth it for us.

Objective

Migrate TrackPoint from Docker Compose to Kubernetes using production-style patterns, reuse your images and pipeline, and produce a recommendation based on your experience.

Requirements

1. Create a cluster: k3s on a VPS (recommended for cost), or kind/minikube locally. Document the choice and its production gaps.
2. Write manifests for all components: Deployments for web, api and worker; a Job for migrations; Services; an Ingress (or Gateway API) with TLS via cert-manager.
3. Configure liveness, readiness and startup probes using `/healthz` and `/readyz`. Explain what each probe does on failure.
4. Set resource requests and limits based on your Stage 9 measurements.
5. Use ConfigMaps and Secrets; Secrets are not committed in plain text (Sealed Secrets, SOPS, or External Secrets).
6. Run PostgreSQL either as a StatefulSet with a PersistentVolumeClaim (learning) or keep it outside the cluster (pragmatic). Justify the choice in an ADR.
7. Configure rolling updates (`maxSurge/maxUnavailable`), a PodDisruptionBudget and graceful termination. Prove zero-downtime deploys under load again.
8. Apply NetworkPolicies so only the API and worker can reach PostgreSQL and Redis.
9. Run pods as non-root with a restrictive `securityContext`; use a dedicated namespace per environment and a least-privilege ServiceAccount for CI.
10. Package with Helm or Kustomize with per-environment overlays; deploy from your CI pipeline.
11. Integrate with monitoring (kube-prometheus-stack or your existing Prometheus) and logging.
12. Write the recommendation: Kubernetes vs Compose for Harborline, with cost, complexity and reliability arguments.

Constraints

CONSTRAINT

Same images and the same pipeline build step as Compose; only the deploy target changes.

The cluster must fit your budget; a single-node k3s cluster is acceptable. Explain what a single node cannot give you.

Break it on purpose

Do these experiments deliberately. Write down what you expected, what actually happened, and why. The gap between the two is where the learning is.

- Give a pod a readiness probe that always fails. What happens to the Service endpoints? To a rolling update?
- Give a pod a liveness probe that checks the database. Stop the database. Watch the cascading restarts, and explain why this is an anti-pattern.
- Request more memory than the node has. Read the scheduler's event message.
- Push an image tag that doesn't exist. Diagnose `ImagePullBackOff` from events.
- Delete the PostgreSQL pod. Did the data survive? Then delete the PVC in staging and learn about reclaim policies the hard way.
- Apply a default-deny NetworkPolicy and fix everything that breaks, one allow rule at a time.

Checkpoints

Do not move to the next stage until every box is ticked **and** you could demonstrate it live.

- ☐ Full stack running on Kubernetes with HTTPS through Ingress and cert-manager.
- ☐ Probes, requests/limits, PDB, securityContext and NetworkPolicies on every workload.
- ☐ Zero-downtime rolling update proven under load.
- ☐ CI deploys to the cluster with a least-privilege identity.
- ☐ Secrets not in plain text in Git.
- ☐ Recommendation ADR written.

Expected outcome

A working, production-style Kubernetes deployment and an informed, honest opinion about when Kubernetes is and isn't worth it.

Prove you understand

Answer these in writing in your `docs/learning-log.md`. If you cannot answer one without searching, you have not finished the stage.

1. Map each Compose concept you used to its Kubernetes equivalent. Where does the mapping break down?
2. What happens, step by step, when you `apply` a changed Deployment? Which controllers act?
3. Liveness vs readiness vs startup probes: give one realistic misconfiguration of each and its consequence.
4. Requests vs limits: how does the scheduler use requests, and what happens when a container exceeds its memory limit versus its CPU limit? What are QoS classes?

5. Why are Kubernetes Secrets only base64-encoded by default, and what did you do about it?
6. Should Harborline move to Kubernetes now? Defend your answer.

Hints (read only when stuck)

HINT — Hint 1 — Debugging order

When a pod misbehaves, look in this order: `get` (status), `describe` (events), `logs` (current and `--previous`), then `exec` into it. Events explain most scheduling and image problems.

HINT — Hint 2 — k3s specifics

k3s ships with its own ingress controller and a local-path storage provisioner. Read what they do, and decide whether to use or replace them.

HINT — Hint 3 — Migrations

A Job for migrations must complete before new API pods take traffic. Research Helm hooks, or ordering in your CD tool, or init containers (and their trade-offs).

Troubleshooting clues

If you see...	Look at...
<code>CrashLoopBackOff</code>	Logs of the previous container; env/config; liveness probe too aggressive
<code>Pending</code> pods	Events: insufficient CPU/memory, unbound PVC, node taints
<code>ImagePullBackOff</code>	Tag exists? Private registry pull secret?
Ingress returns 404	Ingress class, host rules, Service name/port, endpoints
Service has no endpoints	Label selectors vs pod labels; readiness

Study resources

- Kubernetes official docs: Concepts and Tasks sections (especially probes, resource management, NetworkPolicy)
- Kubernetes the Hard Way (Kelsey Hightower), for understanding, not production
- k3s docs; cert-manager docs; Helm or Kustomize docs
- Kubernetes Up & Running (Burns, Beda, Hightower, Evenson)
- learnk8s.io troubleshooting flowchart

PART III — STAGE 11: FINAL INCIDENT CHALLENGE

Everything you built is about to be tested the way production tests it: unexpectedly, at inconvenient times, with incomplete information. This stage is a **game day**. You will face a series of incidents, diagnose them under time pressure, restore service, and write post-mortems.

How to run the game day

1. **Get a fault injector.** Best: ask your mentor for a sealed chaos script that randomly picks and injects incidents into staging without telling you which. Alternative: ask a friend with SSH access to inject them from the list below. Last resort: shuffle the incident cards and have them injected by a script with a random delay so you don't know when or which.
2. **Work only from your tooling.** Your first signal must come from your alerts or external uptime check, not from knowing what was injected.
3. **Run it like a real incident.** Declare the incident, note the start time, keep a timestamped incident log, communicate status updates (write them as if to Jonas in Operations every 15 minutes).
4. **Mitigate first, then fix.** Restoring service (rollback, failover, scale) comes before root-cause analysis.
5. **Write a blameless post-mortem** for at least three incidents, using the template in Appendix B.

CONSTRAINT — Scoring for this stage

Time to detect (TTD): did your monitoring catch it, and how fast?

Time to mitigate (TTM): how quickly were users no longer affected?

Quality of diagnosis: did you find the actual root cause, or just a symptom?

Communication and incident log quality.

Follow-up: did you add an alert, test, guardrail or runbook so it cannot happen silently again?

Incident cards

Each card shows only what you would observe. Clue ladders are for use after the time box; opening a clue costs points in a real assessment, so note which ones you used.

INC-01 — The 9 a.m. 502

Severity	Time box	What you observe
SEV-1	20 min	External uptime alert fires. Tracking page shows "502 Bad Gateway". A deploy completed 4 minutes ago and the pipeline shows green.

Clue ladder (open one at a time, only after the time box):

1. Why did the pipeline say green? What did the smoke test actually check?
2. What do the API container's last 50 log lines say, and how many times has it restarted?
3. Compare the environment variables the new version expects with those provided.

Resolved means: Service restored via rollback; the missing configuration fixed; the smoke test improved so it would catch this.

INC-02 — The certificate nobody watched

Severity	Time box	What you observe
SEV-1	30 min	Customers report browser security warnings on <code>api.</code> only. <code>track.</code> works fine. Nothing was deployed today.

Clue ladder (open one at a time, only after the time box):

1. Check the expiry date of each certificate actually served (not the file on disk).
2. Look at the renewal job's logs. When did it last succeed?
3. What changed recently in firewall or proxy routing that could affect the ACME challenge path?

Resolved means: Valid certificate; renewal proven; the certificate-expiry alert explained (why didn't it fire, or did it?).

INC-03 — Disk at 100%

Severity	Time box	What you observe
SEV-1	30 min	Writes fail. The API returns 500 on parcel creation, reads still work. PostgreSQL logs complain. Disk usage alert fired 40 minutes ago (who was watching?).

Clue ladder (open one at a time, only after the time box):

1. Which directories grew fastest? Think about container logs, images, volumes, WAL.
2. Is a WAL archive or replication slot preventing cleanup?
3. Is there a container writing debug output far faster than normal?

Resolved means: Space recovered safely (without deleting data you need); cause fixed; limits/rotation that make recurrence impossible.

INC-04 — The slow creep

Severity	Time box	What you observe
SEV-2	40 min	p95 latency has risen from 120 ms to 2.8 s over the last hour. Error rate is normal. CPU is moderate.

Clue ladder (open one at a time, only after the time box):

1. Is time spent in the app, waiting for a DB connection, or in the query itself?
2. Look at the slow-query log and active queries/locks in PostgreSQL.
3. Did a recent migration change an index? Has a table grown sharply?

Resolved means: Latency back within SLO; the specific cause identified with evidence (query plan, pool metrics).

INC-05 — OOM roulette

Severity	Time box	What you observe
SEV-2	40 min	Container-restart alert. API replicas restart every few minutes, one at a time. Users see occasional errors. Memory graph looks like a saw tooth.

Clue ladder (open one at a time, only after the time box):

1. What was the exit reason of the restarted containers?
2. Which request types correlate with the memory climb (check logs by route around each restart)?
3. Is a large date range on the reports endpoint loading everything into memory?

Resolved means: Restarts stopped; guardrail added (input limits, streaming, or pagination); limits reviewed.

INC-06 — After the reboot

Severity	Time box	What you observe
SEV-1	30 min	The provider rebooted the host for maintenance. Proxy is up, but every API request returns 503. Grafana is unreachable too.

Clue ladder (open one at a time, only after the time box):

1. Which containers are running and which are not? What are their restart policies?
2. Did the containers start before a dependency (e.g. network mount, volume, Docker itself) was ready?
3. Is the firewall/VPN service up?

Resolved means: Everything running; a reboot test added to your routine; start-up ordering fixed permanently.

INC-07 — The pipeline that won't

Severity	Time box	What you observe
SEV-3	30 min	All pull requests fail CI since this morning with an authentication error during image push. No one changed the workflow file.

Clue ladder (open one at a time, only after the time box):

1. Which step fails, and what credential does it use?
2. Does that credential have an expiry date?
3. Was a third-party action auto-updated because it wasn't pinned?

Resolved means: Pipeline green; credential rotation documented; expiry monitoring or keyless auth (OIDC) considered.

INC-08 — The silent backup

Severity	Time box	What you observe
SEV-2	40 min	The weekly restore test fails: "archive is empty". Backup job reports success every night.

Clue ladder (open one at a time, only after the time box):

1. What does "success" mean in your backup script? Does it check the exit status of every command in a pipe?

2. How big were the last 10 backup files?
3. Did a credential or path change?

Resolved means: Fresh valid backup; script fixed to fail loudly; size/freshness sanity checks added; RPO impact assessed.

INC-09 — Redis gone quiet

Severity	Time box	What you observe
SEV-2	30 min	<code>/readyz</code> returns 503 on some API instances but not others. Label generation jobs are piling up. Nothing obvious in the API logs.

Clue ladder (open one at a time, only after the time box):

1. Can an affected container resolve and reach the Redis host name?
2. Was a network recently renamed or recreated in Compose/K8s?
3. Did Redis restart with a different password or bind address?

Resolved means: All instances ready; queue drained; lost jobs accounted for.

INC-10 — The unexplained CPU

Severity	Time box	What you observe
SEV-1 (security)	60 min	Host CPU pinned at 100% since 02:00. No traffic increase. An unfamiliar process is visible. Outbound network traffic to an unknown IP.

Clue ladder (open one at a time, only after the time box):

1. What service is the process running as, and which container (if any) is it in?
2. Which of your services was reachable from the internet without authentication?
3. Check what changed recently in port publishing and firewall rules.

Resolved means: Contained (isolate, preserve evidence), eradicated, credentials rotated, root cause closed, incident report with a timeline; consider whether a rebuild is safer than a clean-up.

INC-11 — The migration that ate a column

Severity	Time box	What you observe
SEV-1	40 min	After a release, the old API instances still running during the rollout throw errors: "column does not exist". Rollback button pressed; errors continue.

Clue ladder (open one at a time, only after the time box):

1. Why doesn't rolling back the code fix the database?
2. Which migration ran, and was it an expand or a contract step?
3. Can you restore just the affected data without a full restore?

Resolved means: Service restored; data integrity verified; migration review checklist added to PR template.

INC-12 — DNS ghosts

Severity	Time box	What you observe
SEV-2	30 min	After your DR drill, some users reach the new server; others hit the old (dead) IP for hours.

Clue ladder (open one at a time, only after the time box):

1. What TTL did the record have before the change?
2. Are there other records (AAAA, CNAMEs, a second provider) still pointing at the old IP?
3. Did any client or proxy cache DNS indefinitely?

Resolved means: All traffic on the new server; pre-migration TTL lowering added to the DR runbook.

INC-13 — Kubernetes: nothing schedules

Severity	Time box	What you observe
SEV-2	30 min	After a Helm upgrade, new pods stay Pending ; old pods were already terminated, and capacity is reduced.

Clue ladder (open one at a time, only after the time box):

1. What do the pod events say?
2. Did resource requests change in the values file?
3. What do **maxUnavailable** and the PDB allow?

Resolved means: Capacity restored; rollout strategy that cannot drop below the minimum again.

INC-14 — The leaked key

Severity	Time box	What you observe
SEV-1 (security)	45 min	A developer tells you they pushed a branch containing the production database password to a public fork an hour ago.

Clue ladder (open one at a time, only after the time box):

1. What must happen first: removing the commit or rotating the secret?
2. What else could an attacker have done with it in the last hour? Check logs.
3. Why didn't secret scanning catch it (was it a fork? a different pattern?).

Resolved means: Secret rotated with no downtime, access logs reviewed, history cleaned, scanning gap closed.

The final boss: compound incident

CONSTRAINT — Friday, 16:40

Ask your fault injector to trigger **three incidents at once**, for example INC-03 + INC-05 + INC-07. The disk is filling, the API is restarting, and you cannot deploy a fix through the pipeline.

You must decide what to fix first, what to communicate, and whether breaking your own rules (e.g. a manual hotfix) is justified. If you do break a rule, record it and reconcile it with code afterwards.

This is the scenario most likely to come up in a senior-level interview. Your post-mortem for it is the centrepiece of your portfolio.

PART IV — DEVOPS ENGINEER EVALUATION CHECKLIST

Use this checklist to evaluate yourself honestly (or ask your mentor to run it as an oral assessment). Each item has two halves: **doing** it and **explaining why it exists**. Tick an item only if you can do both without notes. A component you can configure but cannot justify is a component you will misconfigure under pressure.

Level	Meaning	Typical role
Beginner	Can build and run things with guidance; understands individual tools	Intern / junior
Intermediate	Can build a working, automated system alone; understands how tools connect	Junior → mid DevOps
Advanced	Designs for failure, security and scale; makes trade-offs with evidence	Mid DevOps / SRE
Production-ready	Operates systems under real pressure; others can rely on their work and documentation	Mid → senior; on-call ready

Beginner

- ☐ Navigate Linux confidently: files, permissions, processes, services, logs, disk and memory, and explain what each command reveals.
- ☐ Explain why the app runs as an unprivileged user and what systemd adds over running a process by hand.
- ☐ Use Git branches, pull requests and conflict resolution, and explain why `main` is protected.
- ☐ Write a Dockerfile and explain every instruction, image layers and the build cache.
- ☐ Explain the difference between images, containers, volumes and networks, and where data actually lives.
- ☐ Start the full stack with Compose and explain how services find each other by name.
- ☐ Explain what a reverse proxy does and why only it publishes ports.
- ☐ Explain what DNS records you created and what TTL means.
- ☐ Explain why secrets never go in Git, and what `.env.example` is for.
- ☐ Read application and container logs to find the reason a service failed to start.

Intermediate

- ☐ Walk through the full request path (DNS → TLS → proxy → API → DB) and name what can fail at each hop.
- ☐ Explain how ACME certificate issuance and renewal work and how you would know if renewal failed.
- ☐ Build a CI pipeline that tests, scans, builds once and promotes, and explain why rebuilding per environment is dangerous.
- ☐ Explain tags vs digests and why production deploys reference an immutable version.
- ☐ Provision servers with Terraform/OpenTofu and explain state, plans, drift and locking.

- ☐ Configure hosts with idempotent Ansible roles and explain what idempotency protects you from.
- ☐ Build dashboards and alerts and explain RED/USE, percentiles and why alerts should be on symptoms.
- ☐ Implement least-privilege DB roles and explain which role runs what, and why.
- ☐ Take, encrypt, ship off-site and restore a backup, and explain why an untested backup is worthless.
- ☐ Explain liveness vs readiness and demonstrate what each does when a dependency fails.

Advanced

- ☐ Achieve measured zero-downtime deploys and explain draining, health checks and graceful shutdown working together.
- ☐ Perform point-in-time recovery and explain WAL, RPO and RTO in terms of your own design.
- ☐ Run expand/contract migrations and explain why database changes are the hardest part of rollback.
- ☐ Scale horizontally with load balancing, caching and pooling, and justify each limit with load-test data.
- ☐ Harden hosts and containers with defence in depth and explain what each layer stops if the previous one fails.
- ☐ Explain the Docker–firewall interaction and prove exposure from outside, not from assumptions.
- ☐ Deploy to Kubernetes with probes, requests/limits, PDBs, NetworkPolicies and non-root security contexts, and explain the scheduler's decisions.
- ☐ Define SLOs and error budgets and explain how they change engineering priorities.
- ☐ Correlate metrics, logs and (bonus) traces to find root causes quickly.
- ☐ Write ADRs that compare real alternatives and state trade-offs honestly.

Production-ready

- ☐ Rebuild the entire production environment from zero within RTO, following your own runbook, with measured results.
- ☐ Detect, mitigate, diagnose and document incidents under time pressure, prioritising mitigation over curiosity.
- ☐ Communicate clearly during incidents with non-technical stakeholders.
- ☐ Write blameless post-mortems whose action items actually prevent recurrence.
- ☐ Handle a security incident: contain, preserve evidence, eradicate, rotate, and report.
- ☐ Balance reliability, security, cost and speed, and defend decisions to a CTO with numbers.
- ☐ Plan capacity: predict the next bottleneck and its trigger point.
- ☐ Produce documentation good enough that another engineer can operate the platform without you.
- ☐ Know when **not** to add complexity (e.g. a reasoned position on Kubernetes for a small team).
- ☐ Continuously improve: every incident, drill and alert tuning session leaves the system better than before.

How the components work together (oral defence)

The final and most important evaluation. Answer each question out loud, as if in an interview, in under five minutes, using your own system as the example. Record yourself.

1. A developer merges a pull request. Describe everything that happens, component by component, until a customer sees the change, including what would stop it and what would undo it.
2. The production server disappears. Describe the next two hours minute by minute. Which artefacts (code, backups, keys, docs) make recovery possible, and where does each one live?
3. An alert fires at 3 a.m.: p95 latency is high. Walk through your investigation using your actual dashboards and logs.
4. Traffic triples next week. What happens to each component, in the order it would break, and what do you change?
5. An attacker obtains the deploy credential. What can they do, what can't they, and how would you notice?
6. Pick any component in your architecture. Remove it. What breaks, and why did you include it in the first place?
7. Where are the single points of failure that remain, and why did you accept each one?
8. What would you change if the budget were ten times larger? What would you **not** change?

CHECKPOINT — Portfolio readiness

When every Intermediate box is ticked, the project is a strong junior portfolio piece. When every Advanced box is ticked and you have your DR drill report and three post-mortems, it is a mid-level portfolio piece. A production-ready tick list plus a clear, recorded oral defence is something most candidates at this level cannot show.

Appendix A — Suggested Repository Layout

A suggestion, not a requirement. What matters is that a newcomer can find things. Justify your structure in the README.

Path	Contents
<code>app/api</code> , <code>app/worker</code> , <code>app/web</code>	Application source, tests, Dockerfiles
<code>deploy/compose/</code>	Base Compose file + environment overrides
<code>deploy/proxy/</code>	Reverse proxy configuration
<code>deploy/k8s/</code>	Helm chart or Kustomize base + overlays
<code>infra/terraform/</code>	Modules and environment stacks
<code>infra/ansible/</code>	Inventories, roles, playbooks, encrypted vars
<code>observability/</code>	Prometheus config, alert rules, Grafana provisioning, dashboards, Loki config
<code>ops/backup/</code>	Backup, retention and restore-test scripts
<code>ops/loadtest/</code>	Load-test scripts and results
<code>.github/workflows/</code> (or <code>.gitlab-ci.yml</code>)	Pipelines
<code>docs/</code>	Architecture, ADRs, runbooks, post-mortems, reports, learning log, traceability matrix

Appendix B — Templates

Runbook template

Section	What to write
Title & alert name	Exactly as it appears in the notification
What it means	One paragraph in plain language: what users experience
Severity & urgency	Wake someone up? Handle in business hours?
First three checks	Where to look first (dashboards, logs, commands), and what normal looks like
Mitigation options	Rollback, restart, scale, failover, with risks of each
Escalation	Who to involve and when
Verification	How you know it's resolved
Last tested	Date and by whom

Post-mortem template (blameless)

Section	What to write
Summary	Two to three sentences: what happened, impact, duration
Impact	Users affected, requests failed, SLO/error budget consumed, data lost (if any)
Timeline	Timestamped: trigger, detection, key decisions, mitigation, resolution
Root cause(s)	Use "5 whys"; look for contributing factors, not a person to blame
Detection	How was it detected? How could it have been detected sooner?
What went well / badly / where we got lucky	Honest reflection
Action items	Owner, due date, type: prevent / detect / mitigate

Architecture Decision Record (ADR) template

Section	What to write
Title & status	e.g. "ADR-004: Use pgBackRest for backups" — Proposed / Accepted / Superseded
Context	The problem and the forces at play (budget, skills, requirements IDs)
Options considered	At least two, with pros and cons
Decision	What you chose
Consequences	What gets easier, what gets harder, what you would revisit and when

Suggested ADR topics

- Reverse proxy choice (Nginx vs Traefik vs Caddy)
- How CI reaches production (push over SSH vs pull agent vs self-hosted runner)
- Secrets management approach
- Backup tooling and RPO strategy
- Zero-downtime deployment strategy on Compose
- Observability stack and retention trade-offs within budget
- PostgreSQL inside or outside Kubernetes
- Kubernetes: adopt now, later, or never?

Appendix C — Study Resource Library

Area	Resources
Linux	The Linux Command Line (Shotts); "How Linux Works" (Ward); man pages; Julia Evans' zines
Networking	"Computer Networking: A Top-Down Approach" (selected chapters); Cloudflare Learning Center
Git	Pro Git (git-scm.com/book); "Oh Shit, Git!?!"
Containers	Docker docs; "Docker Deep Dive" (Poulton); OWASP Docker Security Cheat Sheet
CI/CD	Continuous Delivery (Humble & Farley); GitHub Actions / GitLab CI docs
IaC	Terraform: Up & Running (Brikman); OpenTofu docs; Ansible for DevOps (Geerling)
Observability	Google SRE Book & Workbook (free); Prometheus: Up & Running; Grafana docs
Security	OWASP Top 10; CIS Benchmarks; "Container Security" (Liz Rice)
Databases	PostgreSQL docs (backup, WAL, roles); use-the-index-luke.com
Reliability	SRE Book chapters on SLOs, incident response and post-mortems; "Release It!" (Nygard)
Scaling	Designing Data-Intensive Applications (Kleppmann); k6 docs
Kubernetes	Official docs; Kubernetes Up & Running; learnk8s.io; Kubernetes the Hard Way
Culture	"The Phoenix Project"; "Accelerate" (Forsgren, Humble, Kim); DORA research

Appendix D — Progress Tracker

Fill this in as you go and commit it to [docs/](#). Honest dates are more valuable than fast ones.

Stage	Target date	Done date	Hours	Hints used	Rebuilt from code?	Biggest lesson
1 Foundation						
2 Containerization						
3 Production deploy						
4 CI/CD						
5 IaC						
6 Observability						
7 Security						
8 Reliability						
9 Scaling						
10 Kubernetes						
11 Incidents						

MENTOR — When you're ready

Bring your mentor your repository link, your traceability matrix and your post-mortems. The review will focus on the questions in Part IV, not on whether the site loads. Good luck, and break things often.